

Automated Outreach Pipeline

Production Project Handoff Report

Prepared on 2026-06-07

This report summarizes the complete SDE assignment implementation: a backend pipeline that discovers lookalike companies, finds decision makers, resolves verified emails, generates AI-personalized cold emails, sends approved campaigns, and records lifecycle status.

Area	Status
Source code	Built in outreach-pipeline/
API	FastAPI endpoints implemented
CLI	Typer CLI implemented with approval prompt
Database	SQLAlchemy models and Alembic migration implemented
Queue/cache	Redis and Celery implemented
Docker	Dockerfile and docker-compose.yml implemented
Tests	19 tests passed with 86.20% coverage
GitHub push	Prepared locally; waiting for repository URL or token
Vercel deployment	Requires deployment strategy; see Remaining Work

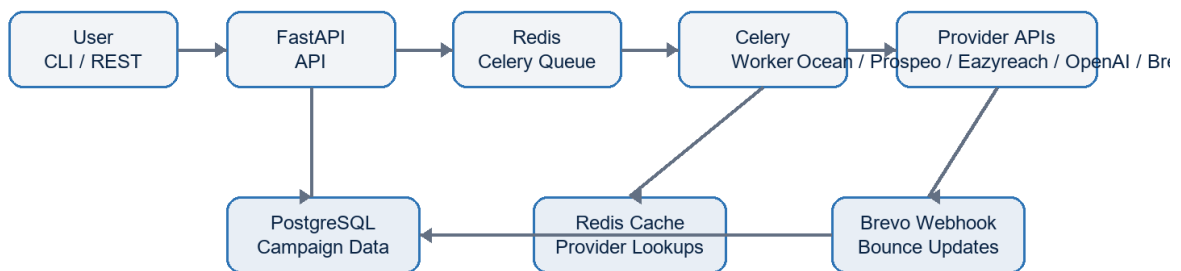
1. Project Overview

The user enters a source company domain such as openai.com. The system discovers similar companies with Ocean.io, finds senior executive and engineering decision makers with Prospeo, verifies emails with Eazyreach, generates personalized outreach with OpenAI, and sends approved emails with Brevo.

- REST API and CLI access are both implemented.
- Campaign, company, contact, and email records are persisted in PostgreSQL.
- External API calls use retries, exponential backoff, rate-limit handling, circuit breaking, Redis caching, and structured JSON logs.
- Docker Compose starts the app, worker, Postgres, and Redis services.

2. Architecture

Automated Outreach Pipeline Architecture



Reliability: 3 retries with exponential backoff, 429 handling, circuit breaker, JSON logs, Redis caching

3. End-to-End Workflow

- 1 User submits a source domain through POST /api/v1/pipeline/start or the outreach CLI.
- 2 Ocean.io returns similar companies and domains.
- 3 Prospeo searches each company and keeps only Founder, CEO, CTO, VP Engineering, Head of Engineering, and Director of Engineering contacts.
- 4 Eazyreach resolves emails from LinkedIn profiles; only verified emails are saved.
- 5 OpenAI generates a personalized subject and short outreach body for each verified contact.
- 6 The system shows a campaign summary before sending. The CLI prompts Proceed? Y/N, while REST exposes POST /pipeline/{id}/send.
- 7 Brevo sends approved emails and the database tracks PENDING, SENT, FAILED, and BOUNCED statuses.

4. Technical Stack

Layer	Technology
Backend	Python 3.12, FastAPI

Layer	Technology
CLI	Typer
Database	PostgreSQL, SQLAlchemy, Alembic
Queue	Redis, Celery
Caching	Redis
AI	OpenAI Responses API
Email delivery	Brevo SMTP API
Quality	Pytest, Black, Ruff, Mypy
Containerization	Docker, Docker Compose
Configuration	Pydantic Settings and .env

5. Database Schema

Table	Key Fields	Purpose
companies	id, name, domain, industry, created_at	Stores lookalike companies.
contacts	id, company_id, name, title, linkedin_url, email, verified	Stores decision makers and verified email status.
campaigns	id, source_domain, status, summary, error_message, created_at	Tracks each pipeline run.
emails	id, campaign_id, contact_id, subject, body, status, sent_at	Tracks generated and sent emails.

6. REST API

Endpoint	Purpose
POST /api/v1/pipeline/start	Create a campaign and queue discovery.
GET /api/v1/pipeline/{id}	Read campaign status, summary, and generated emails.
POST /api/v1/pipeline/{id}/send	Approve sending for generated pending emails.
GET /api/v1/companies	List stored companies.
GET /api/v1/contacts	List stored contacts.
GET /api/v1/campaigns	List campaigns.
GET /api/v1/emails	List generated emails.
POST /api/v1/webhooks/brevo	Receive bounce events from Brevo.

7. Reliability, Security, and Observability

- Retry policy: 3 retries with exponential backoff of 1s, 2s, and 4s.
- Rate-limit handling: HTTP 429 retries automatically and respects Retry-After when present.
- Circuit breaker: repeated provider failures open a provider-specific breaker and recover after cooldown.
- Secrets: API keys are read from environment variables and are never hardcoded.
- Validation: domains are normalized and validated before pipeline execution.
- Logs: structured JSON logs are emitted to stdout and logs/app.log.
- Caching: Ocean, Prospeo, and Eazyreach lookups use Redis cache to reduce provider cost and rate-limit pressure.

8. Running the Project

Local setup:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
alembic upgrade head
uvicorn app.main:app --reload
celery -A app.tasks.celery_app worker --loglevel=info
```

Docker setup:

```
Copy-Item .env.example .env
docker compose up --build
```

9. Verification Completed

Check	Result
pytest	19 passed
Coverage	86.20%, above the 80% requirement
Ruff	All checks passed
Black	All files formatted
Mypy	No issues found in application code

10. GitHub and Vercel Status

Destination	Current Status	Needed From User
GitHub	Not pushed yet. The local folder is ready to initialize, commit, and push.	Provide a GitHub repository URL with write access, or a GitHub personal access token / GitHub CLI login.
Vercel	Vercel CLI is logged in as sm5963397-9665. No production deploy was attempted because this backend requires Postgres, Redis, and a Celery worker.	Choose deployment shape: Vercel API-only plus external DB/Redis/separate worker, or deploy full Docker stack to Render/Railway/Fly/AWS instead.

Destination	Current Status	Needed From User
Provider live run	Not executed against live providers.	Provide Ocean, Prospeo, Eazyreach, OpenAI, and Brevo API keys plus sender domain configuration.

11. Remaining Work / Not Done Yet

- GitHub push is not done because no remote repository or GitHub credential is available in the project folder.
- Vercel production deployment is not done because the complete backend is Docker/Postgres/Redis/Celery-based; Vercel alone cannot run the full worker and infrastructure stack.
- Live provider validation is not done because real API keys were not provided.
- Brevo sender-domain production readiness is not confirmed; SPF, DKIM, DMARC, and warm-up should be configured before real outreach.
- Eazyreach lookup path is configurable because account/version-specific endpoint paths may differ; confirm the exact path from your Eazyreach account if needed.

12. What I Need From You

- 1 GitHub repository URL, for example <https://github.com/<username>/automated-outreach-pipeline.git>, or GitHub token/CLI login with repo write access.
- 2 Decision on deployment target: Vercel API-only with external services and separate worker, or full Docker deployment on a platform that supports long-running services.
- 3 Production environment variables: Ocean, Prospeo, Eazyreach, OpenAI, Brevo, PostgreSQL, Redis, and sender email settings.
- 4 Brevo verified sender/domain details before enabling real email sends.

13. Project File Structure

- app/api: FastAPI routes, middleware, and dependencies
- app/core: settings, security, logging, and application errors
- app/database: SQLAlchemy base and session factory
- app/models: company, contact, campaign, and email ORM models
- app/repositories: persistence access layer
- app/services: external provider clients and shared HTTP reliability utilities
- app/workflows: pipeline orchestration
- app/tasks: Celery application and background tasks
- app/cli: Typer commands
- app/tests: repository, service, API, and workflow tests
- migrations: Alembic environment and initial schema migration

14. Final Build Assessment

The software project itself is built and locally verified. The remaining items are deployment/account tasks, not missing source-code files. Once GitHub and production infrastructure details are provided, the project can be pushed and deployed with environment-specific configuration.